Summary.

1.05 01 00 Modeling Practices

Good Object-oriented modeling starts with keeping class variables private to protect encapsulation, preferring flexible aggregation over inheritance and programming to interfaces to cut down or unnecessary dependencies. Conceptual modeling helps capture real-world ideas and system design across different development phases, while clean code relies on meaningful name conventions and executable unit tests for catching desig flaws early.

1.05 07 Introduction to UML

The Unified Modeling Language gives us an expresive, semi-formal way to model object-oriented software through structural and behavioural diagrams. We can look at these models from three main perspectives: analysis and understanding, the specification for defining software boundaries, and implementation for actual code construction. Following the best practices like keeping diagrams readable, minimizing line crossing, and focussing each diagram on a single core concept makes the entire model easier to maintain.

CamScanner

DÍA MES AÑO NOTA /

Basic Use Cases (1.05 08)

Use cases help figure out what a system needs to do by identifying external actors and their core goals, distinct from step-by-step user interactions. We map these out using use case diagrams, tying everything together with generalization, include, and extend dependencies to break down system behaviour and handle optional variations. Identifying actors and goals helps capture who needs system functionallity and provides a solid basis for incremental scheduling and testing.

Class Diagrams (1.05 09)

Class diagrams dive deeper by showing the types of objects in a system, their attributes, operations, and how they relate to each other through associations, generalizations, aggregations, compositions and dependencies. Multiplicity constraints are crucial here because they define exact cardinality rules for how many objects instances can link together. These diagrams provide supporting detail for specific topics, helping developers visualize the static design view of an application.

UML Quick Reference Guide (1.05 16)

Keeping a handy reperence for UML helps maintain consistency when switching between different diagram types like activity diagram, sequence and communication interaction diagrams, and state charts. Each modeling component has specific best practices, such as ensuring guards and triggers are mutually

DIA MES AÑO NOTA /

exclusive in state charts, properly defining execution occurrences and messages in interaction diagrams, and using correct control flows and nodes in activity diagrams.

CamScanner